

1. Introduction

This project focused on developing two machine-learning systems:

- (1) a general numeric classifier, and
- (2) a spam email detection model using text-based features.

The primary goals were to automate preprocessing, evaluate multiple algorithms, and generate final output files, including four classification results (KhanClassification1–4.txt) and one spam detection output (KhanSpam.txt). Python, scikit-learn, Pandas, NumPy, and TF-IDF vectorization were the primary tools.

2. Data Preparation

2.1 Automated File Handling

All datasets were placed in organized folders (classification/ and Spam Email Detection/). A unified loader script detected CSV, TXT, and XLSX files automatically. This eliminated manual formatting and ensured that every dataset was processed identically.

2.2 Cleaning & Normalization

Datasets often included missing values, non-numeric noise, and inconsistent scales.

The preprocessing pipeline included:

- Mean imputation for missing numeric entries
- Removal of invalid values (NaN, inf, 1e+99)
- StandardScaler normalization, ensuring equal weight across features

Normalization was particularly important for KNN and SVM, which rely on distance-based or margin-based calculations.

2.3 Text Vectorization (Spam Detection)

Spam detection required converting raw email text into meaningful numeric vectors.

The process included:

1. Tokenization and lowercase conversion
2. Stop-word removal
3. Punctuation stripping
4. TF-IDF vectorization

TF-IDF assigns greater importance to unique or spam-triggering terms while reducing the influence of common filler words. This representation produced a high-dimensional but well-structured feature space suitable for SVM.

3. Classification Models

3.1 Model Trials

Several models were tested in early experimentation:

- Decision Tree: simple and interpretable but unstable and slow on larger datasets
- Neural Network: strong accuracy but required high memory and produced inconsistent results

These models were replaced with more efficient classical algorithms.

3.2 Final Models

The final classification pipeline used:

- Random Forest: robust, reduced overfitting, handled mixed features well
- K-Nearest Neighbors (KNN): performed best after normalization, simple but highly effective

Each dataset was trained, evaluated, and exported automatically. Weighted F1-score was the primary measure due to class imbalance across datasets.

3.3 Results

KNN achieved the best performance with a weighted F1-score of 0.88, outperforming Random Forest on most datasets. Its strength came from the combination of standardized features and diverse decision boundaries across the datasets.

4. Spam Email Detection

4.1 Approach

The spam detection task required text-based classification. After TF-IDF vectorization, several models were evaluated:

- Logistic Regression: strong baseline performance
- Naive Bayes: fast but underperformed on large vocabularies
- Support Vector Machine (SVM): best performance overall

4.2 Final Model & Performance

SVM produced the most stable and accurate results, achieving an F1-score of 0.92. Its ability to handle high-dimensional sparse data made it ideal for TF-IDF features.

The final predictions were saved into KhanSpam.txt.

5. Evaluation Metrics

All models were evaluated using:

- Accuracy: overall correctness
- Precision: reliability of positive predictions
- Recall: ability to capture positive cases
- F1-score: balanced measure combining precision and recall

For both tasks, F1-score was the most informative metric due to its robustness on unbalanced datasets, particularly in spam detection.

6. Conclusion

This project successfully produced a scalable machine-learning workflow for both numeric and text data. The system automatically ingested, cleaned, normalized, and processed datasets without manual intervention.

Key findings:

- Automated preprocessing significantly improved consistency
- KNN was the most reliable numeric classifier across multiple datasets
- SVM with TF-IDF was the strongest spam detection model, achieving the highest F1-score
- Naive Bayes proved too simplistic for large, high-dimensional text data
- The project delivered all required result files and demonstrated strong generalization across tasks

Overall, the project achieved its objectives and produced efficient, accurate machine-learning models for both structured and unstructured classification tasks.